★ Fundamentals of AI & ML

- **Artificial Intellegence:**

  Ability of a machine to think, learn & make decision like a human.

  Artificial Technology that enable machine & computer.

★ John McCarthy [Father of AI]

- AI is the Science & engineering of making intelligent machines.

- **Result & Knowing**
  - AI is the Study of agents that precieve their enviornment & act rationally.

→ What makes a system "intellegent"?
- Learn from Data.
- Reason logically.
- make decision
- Improve overtime

Ex
Calculator (no learning)
Google search (learn from user).

- **Data Collection**

  AI rely on large set of data which include image, text or sensor readings for training & development

★ Processing & learning

AI uses algorithms to analogy data identify patterns.

- Model Training

AI model is trained using the data, adjusting it's internal setting to improve its prediction.

- Decision making

Once trained, it can use what it has learned to make decisions.

★ Feed back & improvement

It can improve through feed back, espacially in methods like reinforcement learning.

For Example

Application	AI role
Google maps	Predict traffic.
Net flix	Recommend movie.
Voice assistant	understant speech
Email spam filter	classifier - e-mail.

★ History of AI

Phase - 1 : Birth (1950 - 1960).

In 1950, publication of Alan Turnings network "computing Machinary & Intelligence", which introduce Turning Test which measure computer intellegence.

"Can machine think"

In 1956, term (AI) was introduced by John M Carthy (father of AI).

Phase-2 : The Golden year And Early Success (1960 - 1970)

The period saw rapid Enthausism & optimism in field of AI. AI program could play game.

Early natural language processing attempts began & rule based experts system were envisaged.

Phase - 3 : First AI winter (1970 - 1980)

This is the winter era of AI.
- AI system failed to scale or generalise.
- Funding from DARPA & other agencies was
- critics argued, AI progress was exaggerated

★ Phase-4: Expert System & AI Revival (1980-

AI Experienced a resurgence in the 1980's thro
to Experts System.

• • Expert System :→ Program that enumerates decision
- maping abilities of human ex

using If then,

for Eg → MyCIN, a medical diagnosis.

still no learning capability.

•

Phase-5: Rise of Machine Learning (1990s.-20

the 1990's introduced a new paradigm ie Learning
From Data Instead of Hard core Rule.

★ Machine Learning algorithm like Decision tree
SVM and Neural Networks introduced.

IBM's Deep blue (1997) defeated world chess ch
Gary Kasparov.

AI traditioned from logic-heavy to data-d
approaches, setting the stage for today's bi
throughs

Phase-6: Deep Learning & Modern AI (2010-Pr

Big Data + Deep Learning + GPU Acceleration M
AI owes its success to three key drivers.

• Big Data:→ Exploration of internet & sensor da

• Deep learning:→ Neural network with many lay
inisating the human brains

became practical & accurate in image classification, NLP etc...

- GPU :- Graphic card accelerated deep learning models.

★ Major Milestones:

Image Net (2012)

Deep learning model by Geoffrey Hinton's team won the competetion, revolutionizing computer vision.

Google DeepMind's Alpha Go (2016)

Defected world champion in GO - a game considerd too complex for AI.

Open AI's GPT Model (2018-2024)

Transforms natural language processing with generative AI capabilities.

★ Artificial Intelligence: It is basically the mechanism to incorporate human intelligence into machines through a set of rules (algorithm). The system do not learn, but still act intelligently.

★ Machine Learning: It is a subset of AI that enables machines to learn from data without being explicitly programmed.

★ **Deep learning**: It is a basically subset of machine learning which makes use of neutral network (similar to the neurons working in our brains) to mimic human brain like behaviour. DL works on large sets of data when compared to ml.

→ **Deep Learning focused on**:

- Images.
- Speech
- Natural language.
- complex patterns.

→ **Types of Machine Learning**

- **Supervised Learning**: In this, the model gets trained on a "Labeled Dataset" (Labeled dataset have both input & output parameters).

- **Types of Supervised learning**:

  – classification: output is categorical.

  Examples:

  - Email Spam detection
  - Disease prediction (Yes/No).

  – Regression: output is a continuous value.

  – Example:

  - House Price Prediction.
  - Temprature prediction.

- How it works:
- Give input Data.
- Give correct output (Yes/No)
- model learns mapping
- Predict for new data.

- Algorithms: Linear Regression, Logistic Regression, KNN, Decision Tree etc.

- UnSupervised Learning

It works on with unlabeled Data, meaning there are no predefined outputs.

- The algorithm finds hidden Pattern, groups or relationships within the data on its own.

→ How it Works:

- Model find Pattern.
- Groups Similar Data.
- No Supervision.

→ Algorithms:

- K-Means Clustering.
- Hierarchical clustering.
- Apriori.

→ Reinforcement Learning: Trains an agent on a sequence of decision through trail & error.

The agent interact with environment, receives feedback in the form of reward or penalties & learn optimal actions overtime.

Example:→ An AI agent learning to play chess gets (+ve) feedback for good moves & (-ve) feedback for bad ones. Overtime, it learns strategies to win more often.

- Here are some of most common reinforcement learning algorithms.
  - Q-learning.
  - SARSA (State-Action-Reward-State-Action).
  - Deep Q-Learning.

- Here are some applications of reinforcement learning
  - Gaming & Simulation.
  - Robotic & automation.
  - Autonomous vehicle.

→ Semi-Supervised Learning: Combines both Supervised and unSupervised approaches.

- It uses a small set of labeled data and a large set of unlabeled data for training useful when labeling is costly or time-consuming.

→ Self-Supervised Learning (SSL): Is a machine approach where models generate their own labels from raw data.

- It doesn't rely on manual annotation but the model learns by predicting parts of data from other parts.

- Example :→ In NLP, models like BERT or GPT learn by predicting masked words in sentences, using surrounding content as supervision.

★ Applications and Impact of AI:

- AI finds extensive applications across various sectors including E-commerce, Education, Robotics, Health care, Social media, and many more.
- Below is the list of including those areas:

E commerce:

→ Application

1) Recommendation System
   - AI suggests product based on
   - user search history.
   - previous purchase
   - likes & clicks.

   e.g. "Recommended for you"

2) Chat bots & Virtual Assistants:

   AI chatbots provide 24×7 customer service
   - Answer customer question.
   - Show order, status.
   - Help in return & refund.

→ Impacts on AI in ecommerce

→ Positive Impact

→ Increased Sales
→ Better customer experiance.
→ Reduced cost.

→ Negative Impact

→ Job Loss.
→ Data Privacy issue.

Eg Amazon, Flipkart, Myntra.

• Social Media

→ Application

→ Content Recommendation System.
• Vedios suggest.
• Reels Insta.
• Post on facebook

→ Face Recognisation & filters

face unlock.
Camera beauty filter.

→ Impact

(+ve) → Better user Exp., better security, Time s.
(-ve) → Addiction problem, fake news spread.

• Education Purpose

→ Application

1) Personalized Learning
2) Virtual Teacher Chatbox.
3) Smart content creation.

→ Impact

1) Better Learning Experience.
2) Time Saving for teachers.
3) Education for everyone.

• Robotics

→ Application

1) Industrial Robots.
2) Medical Robots
3) Service Robots.

→ Impact

1) Increase Efficiency.
2) Reduce human error.

a) Job loss
b) high cost
c) Dependence on machine

• GPS & Navigation:

→ Application

1) Smart Route Planning.
2) Real time traffic prediction.
3) Voice Navigation
4) Self driving vehicles.

→ Impact

1) Saves Times.
2) Reduce fuel cost.
3) Improves Safety.

4) Over dependence on GPS.
5) Privacy Issues.
6) Wrong Direction Sometime.

→ Health Care

→ Application

1) Disease Diagonosis.
2) Medical Imaging.
3) Robot Surgery.
4) Drug Discovery.

→ Impact

1) Early & Accurate Diagnosis.
2) Better ~~bet~~ Patient Care.
3) Saves time & cost.

4) High cost of AI System.
5) Data Privacy Issue.
6) Over dependence on machine.
7) Risk on technical Errors.

Automobiles

→ Application

1) Self - driving cars.
2) Advanced driver Assistant System.
3) Voice control & Smart Assistant

→ Impact

1) Improved safety.
2) Better driving Experience.
3) Save fuel & Time.
4) High cost
5) Job loss
6) Cyber security Risk.

Life Styles

→ Application

1) Smart phones & Voice Assistance.
2) Smart Home.
3) Entertainment.
4) Health & fitness.

→ Impact

1) Save time.
2) Better health & safety.
3) makes life comfortable.
4) mobile & Internet Adiction
5) Privacy issue.
6) Less physical activity.

• Gaming

→ Application

1) Smart game character.
2) Realistic movement & Behavior.
3) Difficulty level adjustment.

→ Impact

1) More realistic Games
2) Better player Experience.
3) Addiction
4) Unfair advantage.

• Chatbots

→ Application

1) Customer Support.
2) Banking & finance.
3) Education.
4) Healthcare.

→ Impact

1) 24X7 Service
2) Save Time
3) Reduces company cost
4) Limited understanding.
5) No human Emotions.
6) Technical problem.

- Surveillance

→ Application

1) Face Recognition.
2) CCTV Smart monitoring.
3) Traffic monitoring.
4) Crowd monitoring.

→ Impact

1) Better Security.
2) Save huma effort.
3) faster response.
4) Privacy Issues.
5) Data misuse.
6) Wrong Identification.

- Data Security

→ Application

1) Threat Detection.
2) fraud Detection.
3) Identity verification.
4) Automated security report.

→ Impact

1) faster threat Detection.
2) Reduces Huma effort.
3) Better accuracy.
4) AI can be hacked.
5) high cost.
6) Over Dependance on Technology.

- Entertainment

→ Application

1) Personalized Recomendations.
2) Content creation.
3) Gaming.
4) Voice & Image recognisation.

→ Impact

1) Personalize experience.
2) faster content creation.
3) cost saving.
4) Job loss
5) Privacy concern.
6) false content risk.

★ AI Agents & Enviornments:

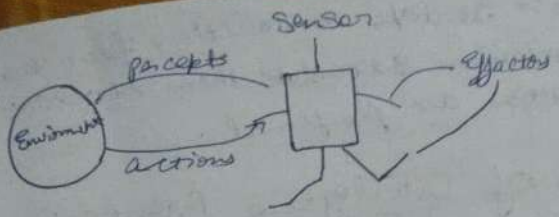
- It is anything that can prosceive its enviornment through Sensors & act upon that enviornment through effactors/ acunators to maximize the performance or achieve some goal.

→ What is Enviornment?

→ The Enviornment is everything outside the agent that the agent intracts with.

→ The agent recieve information from the Enviornment (Input).

→ The agent acts on the Enviornment (output).



- **PEAS**

It stands for Performance measure, Environment, Accurators & Sensors. It is a frame work that is used to describe an AI agent.

- **Performance measure**:→ It is a criteria that measure the success of the agent. It is used to evaluate how well the agent is achieving its goal. for Ex→ in a spam filter system, the performance measure could be minimize the no. of spam emails reaching the inbox.
- **Environment**:→ It represent the domain or context in which the agent operates and interacts.
- **Accurators (Outputs)**:
  - Used to perform action.
  - Ex → wheels of robots, speaker, robotic arm.
- **Sensor (Input)**:
  - Used to precieve the environment.
  - Ex → Camera (Image input), microphone (voice input), Temperature sensor.

• Effactors :→ It takes instructions from decision making mechanism & translate them into actions & those actions are performed.

• Examples of Intelligence Agents :→ Self-driving cars, recommendation system, virtual assistants, & game-playing AI, etc.

• Types of Agents :

• Simple Reflex Agents :→

→ Simple reflex agents act solely on the current percept using predefined condition-action rules, without storing or considering any history.

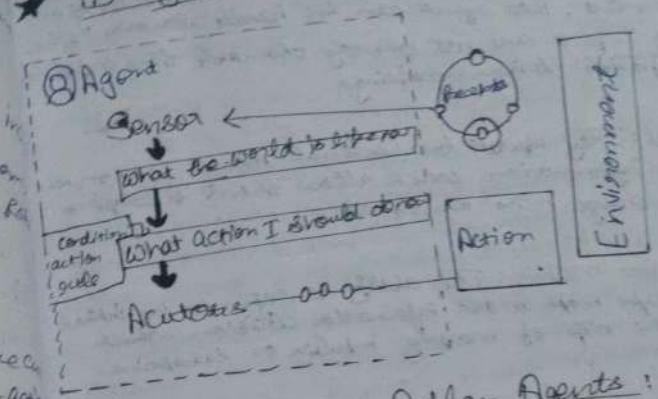
→ They are fast and easy to implement, making them suitable for fully observable, stable environment with clear and simple rules.

However, they tend to fail in dynamic & uncertain because they lack memory and deeper reasoning capabilities.

→ When to Use :→ They are ideal in controlled well defined environments such as basic automation like home automation system or real-time reactive systems like sensor or switch.

→ Example :→ Automatic Door (IF person detected THEN open).

★ Diagram of reflexive



• Model-Based Reflex Agents:

→ Model-Based reflex agents are a type of intelligent agent designed to interact with partially observable or dynamic environments.

→ Unlike simple reflex agents, Model-Based reflex agents maintain an internal state that reflects not just the current perception but also incorporate past observation to infer aspects of the environment that are not directly visible.

→ The key diff. b/w simple reflex agents & model-based agents in AI is that they have memory- they remember what they've seen before.

→ They memory (or internal state) is built based on the history of what the agent has pre

→ By storing this information, the agent can ma more informed decisions because it can pre how the environment might change based o its actions.

• Key characteristics:

— Internal State: By maintaining an internal model of the environment, these agents can handle scenarios where direct observations are not directly observable thus making them flexible decision-making.

— Adaptive: They update their internal model based on new information which allow them to adapt to changes in the environment

— Better Decision making: The ability to refer to the internal model helps make more informed decision which reduces the risk of making impulsive or suboptimal choices.

— Increased complexity: Maintaining an internal model and computational demands which require more memory & processing power to track changes in environment.

• When to Use: They are beneficial in situation where the environment is dynamic & not all elements can be directly observed at once. Autonomous driving cars, robotics & surveillance system are good ex

• Goal Based AI Agents

— Goal Based AI agents are programmed to achieve specific objectives.

— These agents are designed to plan, execute, & adjust their action dynamically to meet predefined goal.

— Goals are the specific objectives that the agents aims to achieve. these can be range from basic tasks. such as sorting objects, to complex mission, such as navigating a robot through a maze, solving a puzzle.

— Planning involves determining the sequence of actions required to achieve the goal. this process can be complex, involving predictive models, heuristic & algorithms to evaluate possible future states & actions.

— Execution is the phase where the agent carries out the planned actions. this involves interacting with the environment & performing tasks that bring the agent closer to its goal.

— Adaptation is essential as the agent interacts with

★ Utility - Based Agents

- They agents make decisions based on utility function
- This function measures the degree of satisfaction or utility associated with different possible outcomes.
- They agent evaluate multiple potential actions & select the one that maximizes their overall utility
- Step-by-Step decision making process:
- Perceive the environment: the agent gathers info about its current state from the environment.
- Generate possible actions: - The agent identifies all possible actions it can take from the current environment.
- Predict outcomes: Using the transition models the agent predicts the resulting state for each possible action.
- Evaluate utility: For each predicted state the agent calculate the utility using the utility fn.
- Select the optimal action: → the agent compares utility values of all predicted states & selects the action that leads to the highest utility.
- Act the observe: The agent perform the selected actions & after acting, the agent observes the new state & update its knowledge about environment.
- Learn & adapt: Based on the outcomes, the agent may update its

★ BFS (uninformed Search / blind Search)

It is a search strategy in which the algorithm has no additional information about the goal state other than problem definition.

Ex → BFS, DFS;

1) BFS (Breadth first Search).

Bfs explore the search tree level by level.
→ first explore all nodes at depth (vertically)
→ then depth 1.
→ So on ——.

It uses Queue (Fifo) data structure (First in first out).

→ Properties of BFS :

It follow fifo (Queue).
Shallowest node.
It is complete algorithm → It give always warranty that they give atleast one sol^n.
It is always assume that it give optimal sol^n.

Time complexity → $O(b^d)$ d → depth.
b → branch factor
O → big-O notation.
bigO mean → worst case.

★ DFS (Depth first Search)

Depth first search explore the deepest node before backtracking.

- → goes deep into one branch.
- → then backtrack.

It follow Stack (LIFO) (Last in first out).

Properties of DFS

- → DFs is not assure that they give complete sol ★ they think may get stuck in infinite loop (depth)
- It is not give optimal sol^n.

Time complexity → $O(b^d)$.

★ Data Structure → Stack (LIFO).

Advantage → Require Less memory than BFS.

★ Informed Search

It is also called heuristic search. it is a search strategy that uses additional information knowledge) (heuristic) to guide the search toward the goal more efficiently.

- → uses problem — specific knowledge,
- → expands fewer nodes.
- → Is faster & more efficient.
- → It does not search blindly, it chooses the most promising path first.

• A*

Tim

heuristic value → It is helping to take more informed decision & reduce the search space.

h(n) → manhattan Distance → $|x_1 - x_2| + |y_1 - y_2|$, ~~h(n) → Euclidian~~

Straight line Distance → Euclidian

$$\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

★ A*

$$f(n) = g(n) + h(n)$$

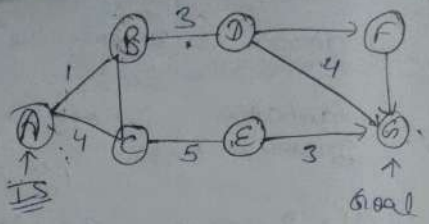
- $g(n)$ → actual cost to node (n)
- $h(n)$ → Estimate cost from the node 'n' to goal.

- A* is complete.

Time complexity :- branching factor reduce drastically.

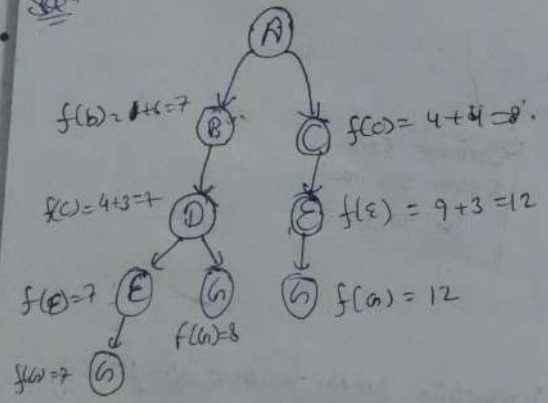
Q2.

A	5
B	6
C	4
D	3
E	3
F	1
G	0



$$f(n) = g(n) + h(n)$$

Soln



Hence the path is $A \to A \to B \to D \to E \to G$.

★ Data Scaling

Data Scaling is a pre processing technique used in ML to adjust the range of feature value so that all the variable contribute equally to model.

many ML Algo. Like (KNN, SVM, linear regression) are the sensitive to feature magnitude. If one feature has value in thousand & another in decimal, the large one dominates.

★ Data Cleaning

Data cleaning is a preprocessing technique. It is the process of to detect, correcting errors, inconsistency, inaccuracies in a dataset to improve qualities before analysis or ML.

Raw data usually noisy. If you train a model on dirty data, you'll get poor results.

→ Importance

- Improve model accuracy.
- Remove noise & errors.
- Make data consistent & Reliable.

★ Forward Fill

It replaces a missing value with the last known (previous) value. It moves forward in dataset

★ Backward Fill

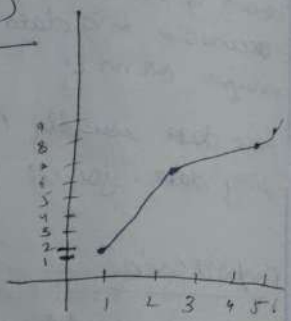
It replaces a missing value with the next known value. It moves backward in dataset

Linear Regression

★ Regression :⇒ continuous value output, comes under supervised learning.

years	~~stock~~ price
2000	190
2002	195
2002	220
⋮	
2026	250

(features) X	Y (label)
1	2
2	7
3	8
4	9
5	



$$y = mx + c$$

$$h(x) = \theta_0 x + \theta_1$$

hypothesis f^n ; m ; c

$$m = \frac{n\Sigma XY - (\Sigma X)(\Sigma Y)}{n\Sigma X^2 - (\Sigma X)^2}$$

$$c = \frac{\Sigma Y - m\Sigma X}{n}$$

MAE

MSE

RMS

MAE (Mean Absolute Error)

$$\frac{\sum_{i=0}^{n} |\hat{y}_i - y_i|}{n}$$

MSE (Mean Square Error)

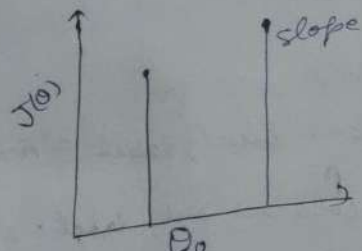
$$\frac{\sum_{i=1}^{n} (\hat{y}_i - y_i)^2}{n}$$

$$= \frac{(2-1)^2 + (2-2)^2 + (4-3)^2 + (4-4)^2}{4}$$

$$= \frac{1+1}{4} = \frac{2}{4} = 0.5$$

RMSE (Root mean square Error)

$\sqrt{0.5}$



$$\frac{\partial J(\theta_0)}{\partial \theta_0}$$

$$\theta_0 \left(\alpha \frac{\partial J(\theta_0)}{\partial \theta_0} \right)$$

learning rate.

★ Linear Regression

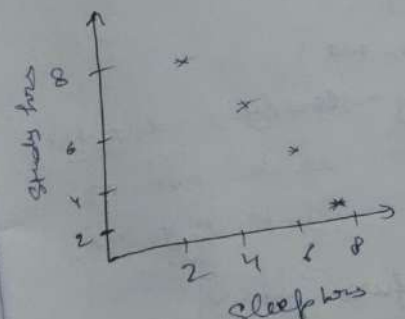
```
import numpy as np.
import matplotlib.pyplot as plt.
from sklearn.linear_model import LinearRegression

# X is a 2D array representing (n_samples, n_features)
X = np.array([[1],[2],[3],[4],[5],[6]])

# Y is a target variable.
Y = np.array([2, 3, 5, 4, 6, 7])

model = LinearRegression().
model.fit(X, Y)
Y_pred = model.predict(X)

# print trained model parameters.
Print (model.coef_)
Print (Model.intercept_)

# Visualizing the result.
plt.scatter (X, Y, color='blue', label='Actual
plt.plot(X, Y_pred, color='red', label='Reg
                                        Line'
plt.xlabel ('Independent variable (X)')
plt.ylabel ('Dependent variable (Y)')
plt.show()
```

★ KNN (K-nearest neighbours)

→ Majority voting } based algorithm.
→ Proximity.
→ Majorly used for practice.

Sleep hrs	Student hrs	result
2	8	Pass
3	7	Pass
6	3	Pass
7	2	

let K=3, predict class of a student whose study hrs = 4 & sleep hrs = 6.



X = feature
y = labels - IO.

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
error = mean_absolute_error(y_pred, y_test)

★ KNN Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_pred, y_test))
print("confusion matrix: \n", confusion_matrix(y_pred, y_test))
```

Accuracy Score

y_test = (1, 0, 1, 1, 0)

y_pred = (0, 0, 0, 1, 0)

acc_score = right_pred / total_pred

= 3/5 = 60%

Confusion matrix.

	predict(0)	predict(1)
Actual(0)	TN = 2	FP = 0
Actual(1)	FN = 2	TP = 1

$\frac{TP+TN}{TP+TN+FN+FP}$ $\frac{2+1}{5} = \frac{3}{5}$

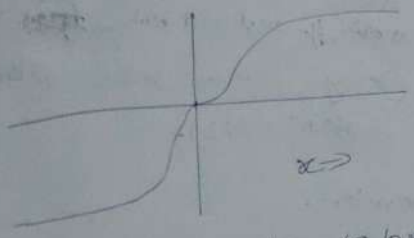
= 0.6

★ Logistic Regression

→ Prediction in form of yes/No
→ Generally used for classification problem
→ The heart of LR is Sigmoid f^n.

LR → $\frac{1}{1+e^{-x}}$ $\sigma(x)$

Sigmoid f^n Generally used to



Sigmoid f^n → Data is close to probability to 0 or 1.
0.5 < True → 1
0.5 > False → 0.

→ from sklearn.model_selection import train_test_split

80% → training
20% → testing

X_train, X_test, y_train, y_test = train_test_split(df[['age']], df.bought_insurance, test_size=0.2)

from sklearn.linear_model import LR
model = LogisticRegression()

$$J = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right)$$

[cross entropy]

Code of Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, recall_score,
                             confusion_matrix, classification_report).

X = [[1,2],[5,7], ... ]
y = [0,1,1,0,1 ... ]
X_train, X_test, y_train, y_test = train_test_split(
                    (X, y, test_size = 0.2, random_state = 42).

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred = model.predict_proba(X_test)[:,1]

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("classification report")
print(classification_report(y_test, y_pred))
```

f1_score = 2 * precision * recall

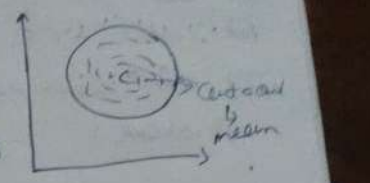
★ K means clustering

unsupervised learning

K ⇒ No. of columns.

[Every clusters have a centroid]

K = 1 to 10 → random



with in cluster sum of square → WCSS / Inertia / error

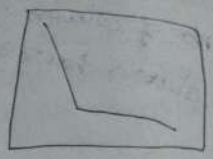
$$= \sum_{K=1}^{10} \sum_{n \in C_i} |(x - \mu_{Ci})|^2$$

$$\sum_{n \in C_1} (n - \mu_{C1})^2 + \sum_{n \in C_2} (n - \mu_{C2})^2$$

★ Code of Silhouette Score

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import KMeans
import matplotlib.pyplot as plt

data = {

X = df[["Values"]].values

WCSS = []
for K in range(1, 11):
    Kmeans = KMeans(n_clusters=K, random_state=4
    Kmeans.fit(X)
    wcss.append(Kmeans.inertia
```

```
plt.plot(range(1,6), wcss, marker='o')
plt.xlabel("No. of clusters K")
plt.ylabel("wcss")
plt.title("Elbow method")
plt.show()
```



Silhouette score → again elbow graph ka aya to y axis range

★ Forward Selection

```
from sklearn.feature_selection import SequentialFeatureSelector
model = LinearRegression()
fs = SequentialFeatureSelector(model, n_features_to_select=5, direction='forward')
fs.fit(X, y)
features_selected = X.columns[fs.get_support()]
```

★ Cross Validation :-

- hold out CV.

↳ train_test_split()

(train 80%) (test: 20%)

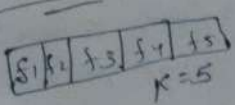
train_test_split

↙ train (70%) ↘ validation (30%)

★ Leave One out CV (LOOCV) :-

$\frac{A1 + A2 + \cdots - An}{n}$

★ K-fold CV



K=5, so fold = 5

n = 10.

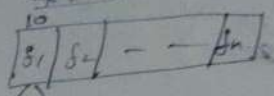
Total iteration = no. of fc.

If K=1, then last fold will have only one sample.

★ Stratified K-fold CV

no. of Sample = n → 80% (Class = 0)
→ 20% (Class = 1)

k = 10



80% (0) 20% (1)

★ SVM (Support Vector Machine)

- It is Supervised Machine learning for classification & Regression.
- It tries to find best boundary – hyperplane
- For doing binary classification.

- Key concept
- Hyper plane :→ $wx + b = 0$

w = weight parameter,
& boundary separating two diff. classes.

- Support vector → The closest data points to hyperplane crucial for determining hyperplane & margin.
- margins → the dist. b/w hyperplane & support vectors.

- Kernal → A f° that maps data to higher-dimensional space enabling SVM handle non-linear classification.
- Hard margins :→ A perfectly separated data by margin
- Soft margin :→

★ Types of SVM

- Linear :→ Used when linearly separable.
- Non-linear :→ Not linearly separable.
- A Kernal is a f° that maps data from its to a higher dimension space when they become linearly separable.

Linear :→ model = SVC (kernal = 'linear')

Polynomial :→ model = SVC (Kernal = 'poly', degree = 3)

Radial basis f° :→ Transform data into a space based on distance b/w data base.

model → model = SVC (kernal = 'rbf', gamma = 0.5, c = 1)

★ Mathematical computation

- consider binary [1, -1]
- we have a training data set consisting of input features vector x & their classes y.
- Eg → $w^T x + b = 0$.
- marginal plane
  - positive → $w^T x + b = 1$
  - negative → $w^T x + b = -1$

★ Linear SVM classifier

$$\hat{y} = \begin{cases} 1 & w^T x + b \geq 1 \\ -1 & w^T x + b < -1 \end{cases}$$

$\hat{y}$ is a predictable label of data point.

- $y_i (w^T x_i + b) \geq 1$ for $i = 1, 2, 3, \ldots m$
  (assume data point is correctly classified)

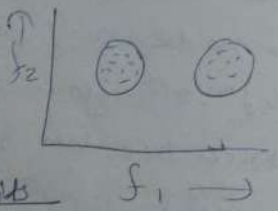
★ Working of SVM

- Data plotting.
- Support vectors identification.
- marginal plane creation.
- identify the most optimal hyperplane.
- Kernal Trick.

★ Silhouette Score :-

→ Cohesion

measure of how close a data point is with its neighboring points within its own clusters.



→ Separation :- measure of the distance b/w a point data point in a cluster with all data points of other clusters.

Cohesion a(i) = a(i) = Avg euclidean dist. of a particular data point in a cluster with all data points in that cluster.

Separation (b(i)) b(i) ⇒ Avg euclidean dist. of a particular data point 'i' with all data points of another cluster.

{Note:- a(i) is more & b(i) is less then cluster is good.

Silhouette score

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

value range from -1 to 1.

if S(i) = -ve(-1) bad cluster),

S(i) = +ve(+1) (ohdiatly proper clustering).

$-1 \leq S(i) \leq 1$

poor cluster

good cluster.